

EXP 4

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```
import numpy as np

# Input Dataset (Hours Studied, Attendance)
X = np.array([
    [2, 60],
    [3, 65],
    [5, 75],
    [6, 80],
    [8, 90]
], dtype=float)

# Output Dataset (Pass=1, Fail=0)
y = np.array([[0], [0], [1], [1], [1]], dtype=float)

# Normalize Input
X = X / np.amax(X, axis=0)

# Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of Sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)
```

```
# Initialize Weights

np.random.seed(1)

input_neurons = 2
hidden_neurons = 3
output_neurons = 1

W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
W2 = np.random.uniform(size=(hidden_neurons, output_neurons))

# Learning Rate

lr = 0.5

# Training using Backpropagation

epochs = 5000

for i in range(epochs):

    # Forward Propagation

    hidden_input = np.dot(X, W1)
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2)
    predicted_output = sigmoid(final_input)

    # Error

    error = y - predicted_output

    # Backpropagation
```

```

d_output = error * sigmoid_derivative(predicted_output)

hidden_error = d_output.dot(W2.T)
d_hidden = hidden_error * sigmoid_derivative(hidden_output)

# Update Weights
W2 += hidden_output.T.dot(d_output) * lr
W1 += X.T.dot(d_hidden) * lr

print("Training Completed")

print("\nPredicted Output:")
print(predicted_output)

# Test New Sample
test = np.array([[7, 85]], dtype=float)
test = test / np.array([8, 90])

hidden = sigmoid(np.dot(test, W1))
output = sigmoid(np.dot(hidden, W2))

print("\nTest Sample: Hours = 7, Attendance = 85")

if output[0][0] >= 0.5:
    print("Prediction : PASS")
else:
    print("Prediction : FAIL")

```

```
print("Output Value =", output[0][0])
```

Output :

```
>>> ===== RESTART: C:/Users/yuvas/Downloads/win-g20:
Training Completed

Predicted Output:
[[0.00186658]
 [0.03740913]
 [0.96300501]
 [0.99565248]
 [0.99943047]]

Test Sample: Hours = 7, Attendance = 85
Prediction : PASS
Output Value = 0.9988298700926677
>>>
```